

JSONB Mastery in PostgreSQL

Table of Contents

- [1. Learning Objectives](#)
 - [2. Introduction to JSON vs JSONB](#)
 - [3. Storage and Internals](#)
 - [4. Core Operators — Complete Reference](#)
 - [5. 20+ Realistic Operator Examples](#)
 - [6. JSONB Functions](#)
 - [7. GIN Indexing for JSONB](#)
 - [8. JSONPath Queries](#)
 - [9. Modifying JSONB Data](#)
 - [10. Real-World Use Cases](#)
 - [11. Common Mistakes](#)
 - [12. Best Practices](#)
 - [13. Performance Considerations](#)
 - [14. Interview Questions & Answers](#)
 - [15. Exercises and Solutions](#)
 - [16. Cross-References](#)
-

Learning Objectives

By the end of this module you will be able to:

- Explain the internal difference between `json` and `jsonb` and when to choose each
 - Use all 9 JSONB operators fluently with realistic query patterns
 - Build GIN indexes on JSONB columns and understand which operator classes apply
 - Write JSONPath expressions for complex nested traversals
 - Design schemas that blend relational and JSONB columns effectively
 - Diagnose performance issues in JSONB-heavy workloads
-

Introduction to JSON vs JSONB

PostgreSQL provides two JSON data types:

Feature	<code>json</code>	<code>jsonb</code>
Storage	Exact text copy	Decomposed binary
Duplicate keys	Preserved (last wins on read)	Last key wins, stored once
Key ordering	Preserved	Not preserved
Write speed	Faster (no parsing)	Slower (parse + decompose)
Read/query speed	Slower (re-parse every time)	Faster (pre-parsed)
Indexing	Not supported	GIN / btree-gin
Operators	Subset	Full set

Whitespace	Preserved	Stripped
------------	-----------	----------

Rule of thumb: Use `jsonb` for almost everything. Use `json` only when you need exact round-trip fidelity (e.g., storing audit logs where key ordering matters).

```
-- Demonstrating the difference
SELECT '{"a":1,"a":2}':::json; -- returns {"a":1,"a":2} (both keys kept)
SELECT '{"a":1,"a":2}':::jsonb; -- returns {"a": 2} (deduplication)

SELECT '{"b":2, "a":1}':::json; -- returns {"b":2, "a":1} (order preserved)
SELECT '{"b":2, "a":1}':::jsonb; -- returns {"a": 1, "b": 2} (alphabetical)
```

Storage and Internals

JSON text: `'{"user_id": 42, "tags": ["admin","read"], "meta": {"region":"us-east"}}'`

JSONB internal binary layout:

JSONB Container Header

JB_FOBJECT | nPairs=3 | JEntry array (offset + type per key)

Keys (sorted): "meta" "tags" "user_id"

Values: {object} [array] 42

Nested object "meta":

JB_FOBJECT | nPairs=1 | "region" → "us-east"

Because JSONB stores keys in sorted order, key lookups are $O(\log n)$ binary searches rather than $O(n)$ linear scans. This is why JSONB reads are faster despite being slower to write.

Core Operators — Complete Reference

Navigation Operators

Operator	Description	Returns
<code>-></code>	Get JSON object field by key	<code>jsonb</code>
<code>->></code>	Get JSON object field as text	<code>text</code>
<code>#></code>	Get nested value by path array	<code>jsonb</code>
<code>#>></code>	Get nested value by path array as text	<code>text</code>

Containment Operators

Operator	Description
@>	Left contains right (does left document contain all key/value pairs of right?)
<@	Right contains left (is left a subset of right?)

Key Existence Operators

Operator	Description
?	Does key exist at top level?
	Do ANY of the given keys exist?
?&	Do ALL of the given keys exist?

20+ Realistic Operator Examples

Setup: Sample Tables

```
-- E-commerce product catalog
CREATE TABLE products (
  id          SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  attributes  JSONB NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT now()
);

INSERT INTO products (name, attributes) VALUES
('iPhone 15 Pro', '{
  "brand": "Apple",
  "category": "smartphone",
  "price": 999.99,
  "specs": {"ram": "8GB", "storage": "256GB", "cpu": "A17 Pro"},
  "colors": ["black","white","titanium"],
  "in_stock": true,
  "ratings": {"avg": 4.7, "count": 1823},
  "tags": ["premium","5g","flagship"]}'),
('Samsung Galaxy S24', '{
  "brand": "Samsung",
  "category": "smartphone",
  "price": 799.99,
  "specs": {"ram": "8GB", "storage": "128GB", "cpu": "Snapdragon 8 Gen 3"},
  "colors": ["black","purple","cream"],
  "in_stock": true,
  "ratings": {"avg": 4.5, "count": 2341},
  "tags": ["android","5g","amoled"]}'),
```

```

('Sony WH-1000XM5', '{
  "brand": "Sony",
  "category": "headphones",
  "price": 349.99,
  "specs": {"driver": "30mm", "battery": "30h", "anc": true},
  "colors": ["black","silver"],
  "in_stock": false,
  "ratings": {"avg": 4.8, "count": 3102},
  "tags": ["wireless","anc","premium"]
}'),
('Dell XPS 15', '{
  "brand": "Dell",
  "category": "laptop",
  "price": 1499.99,
  "specs": {"ram": "16GB", "storage": "512GB", "cpu": "Intel i7-13700H", "gpu":
"RTX 4060"},
  "colors": ["silver"],
  "in_stock": true,
  "ratings": {"avg": 4.6, "count": 892},
  "tags": ["oled","productivity","gaming"]
}');

-- User events for behavioral analytics
CREATE TABLE user_events (
  id          BIGSERIAL PRIMARY KEY,
  user_id     INT NOT NULL,
  event_type  TEXT NOT NULL,
  payload     JSONB NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT now()
);

INSERT INTO user_events (user_id, event_type, payload) VALUES
(101, 'purchase', '{"product_id": 1, "amount": 999.99, "currency": "USD", "promo":
null}'),
(101, 'search',   '{"query": "wireless headphones", "filters": {"price_max":
400}}'),
(102, 'purchase', '{"product_id": 3, "amount": 314.99, "currency": "USD", "promo":
"SAVE10"}'),
(103, 'pageview', '{"page": "/products/laptop", "referrer": "google", "session":
"abc123"}'),
(104, 'purchase', '{"product_id": 4, "amount": 1499.99, "currency": "USD", "promo":
null}');

```

Example 1: -> — Get field as JSONB

```

-- Get specs object (returns jsonb)
SELECT name, attributes -> 'specs' AS specs
FROM products;

```

```
-- Result for iPhone:  
-- {"cpu": "A17 Pro", "ram": "8GB", "storage": "256GB"}
```

Example 2: `->>` — Get field as text

```
-- Get brand as plain text (for string comparisons, LIKE, etc.)  
SELECT name, attributes ->> 'brand' AS brand  
FROM products  
WHERE attributes ->> 'brand' = 'Apple';
```

Example 3: `->` chained for nested access

```
-- Get average rating (still jsonb)  
SELECT name, attributes -> 'ratings' -> 'avg' AS avg_rating  
FROM products  
ORDER BY (attributes -> 'ratings' -> 'avg')::numeric DESC;
```

Example 4: `->>` on nested object

```
-- Get CPU as text from nested specs  
SELECT name, attributes -> 'specs' ->> 'cpu' AS cpu  
FROM products  
WHERE attributes -> 'specs' ->> 'ram' = '8GB';
```

Example 5: `#>` — Path navigation with array

```
-- Access deeply nested: attributes.ratings.avg  
SELECT name, attributes #> '{ratings,avg}' AS avg_rating  
FROM products;  
  
-- Access array element: attributes.colors[0]  
SELECT name, attributes #> '{colors,0}' AS primary_color  
FROM products;
```

Example 6: `#>>` — Path to text

```
-- Get storage spec as text  
SELECT name, attributes #>> '{specs,storage}' AS storage  
FROM products;  
  
-- Filter products with 256GB storage  
SELECT name, attributes ->> 'price'  
FROM products  
WHERE attributes #>> '{specs,storage}' = '256GB';
```

Example 7: @> — Containment check (left contains right)

```
-- Find all Apple products
SELECT name FROM products
WHERE attributes @> '{"brand": "Apple"}';

-- Find in-stock products in smartphone category
SELECT name FROM products
WHERE attributes @> '{"category": "smartphone", "in_stock": true}';

-- Find products with a specific tag
SELECT name FROM products
WHERE attributes @> '{"tags": ["5g"]}';
-- Note: @> on arrays checks if the array contains the given elements

-- Find products with specific spec values
SELECT name FROM products
WHERE attributes @> '{"specs": {"ram": "8GB"}}';
```

Example 8: <@ — Reverse containment

```
-- Is this small JSON a subset of the product's attributes?
SELECT name FROM products
WHERE '{"brand": "Sony", "in_stock": false}' <@ attributes;
-- Returns Sony WH-1000XM5

-- Practical use: check if user preferences match a product
SELECT name FROM products
WHERE '{"category": "smartphone"}'::jsonb <@ attributes;
```

Example 9: ? — Key existence

```
-- Find products that have a GPU spec
SELECT name FROM products
WHERE attributes -> 'specs' ? 'gpu';
-- Returns Dell XPS 15

-- Find events with a promo code
SELECT user_id, payload ->> 'promo'
FROM user_events
WHERE event_type = 'purchase'
  AND payload ? 'promo'
  AND payload ->> 'promo' IS NOT NULL;
```

Example 10: ?| — Any key exists

```

-- Find products in either laptop or headphones category
-- (Using array of keys to check tag existence)
SELECT name FROM products
WHERE attributes -> 'tags' ?| ARRAY['gaming', 'anc'];
-- Returns Dell XPS 15 (has 'gaming') and Sony WH-1000XM5 (has 'anc')

-- Find events that have either 'promo' or 'referrer'
SELECT user_id, event_type FROM user_events
WHERE payload ?| ARRAY['promo', 'referrer'];

```

Example 11: ?& — All keys exist

```

-- Find products where specs has BOTH ram and storage
SELECT name FROM products
WHERE attributes -> 'specs' ?& ARRAY['ram', 'storage'];

-- Find events with all required fields
SELECT * FROM user_events
WHERE event_type = 'purchase'
      AND payload ?& ARRAY['product_id', 'amount', 'currency'];

```

Example 12: Combining operators in WHERE clauses

```

-- Premium in-stock smartphones under $1000
SELECT
  name,
  attributes ->> 'price' AS price,
  attributes #>> '{ratings,avg}' AS avg_rating
FROM products
WHERE attributes @> '{"category": "smartphone", "in_stock": true}'
      AND (attributes ->> 'price')::numeric < 1000
ORDER BY (attributes #>> '{ratings,avg}')::numeric DESC;

```

Example 13: Aggregating JSONB fields

```

-- Average price per category
SELECT
  attributes ->> 'category' AS category,
  AVG((attributes ->> 'price')::numeric) AS avg_price,
  COUNT(*) AS product_count
FROM products
GROUP BY attributes ->> 'category';

```

Example 14: Array element access with index

```
-- Get first and last color for each product
SELECT
    name,
    attributes #>> '{colors, 0}' AS first_color,
    attributes -> 'colors' AS all_colors,
    jsonb_array_length(attributes -> 'colors') AS num_colors
FROM products;
```

Example 15: Filtering on numeric JSONB values

```
-- Products with average rating >= 4.7
SELECT name, attributes #>> '{ratings,avg}' AS rating
FROM products
WHERE (attributes #>> '{ratings,avg}')::numeric >= 4.7;

-- Products with more than 2000 reviews
SELECT name
FROM products
WHERE (attributes #> '{ratings,count}')::int > 2000;
```

Example 16: Using JSONB in JOINS

```
-- Join user_events to products using JSONB field
SELECT
    e.user_id,
    p.name AS product_name,
    (e.payload ->> 'amount')::numeric AS amount_paid
FROM user_events e
JOIN products p
    ON (e.payload ->> 'product_id')::int = p.id
WHERE e.event_type = 'purchase';
```

Example 17: UPDATE with JSONB operators

```
-- Update a nested field using jsonb_set
UPDATE products
SET attributes = jsonb_set(
    attributes,
    '{specs,storage}',
    '"512GB"'
)
WHERE name = 'iPhone 15 Pro';

-- Add a new top-level field
UPDATE products
SET attributes = attributes || '{"warranty_years": 2}'
WHERE attributes ->> 'brand' = 'Apple';
```



```
-- Remove a key
UPDATE products
SET attributes = attributes - 'warranty_years'
WHERE attributes ->> 'brand' = 'Samsung';
```

Example 18: Unnesting arrays with jsonb_array_elements

```
-- Expand tags into rows
SELECT name, tag
FROM products,
     jsonb_array_elements_text(attributes -> 'tags') AS tag
ORDER BY name, tag;

-- Count how many products use each tag
SELECT tag, COUNT(*) AS usage
FROM products,
     jsonb_array_elements_text(attributes -> 'tags') AS tag
GROUP BY tag
ORDER BY usage DESC;
```

Example 19: Building JSONB dynamically

```
-- Build a JSONB response object from relational data
SELECT jsonb_build_object(
    'id',      p.id,
    'name',    p.name,
    'brand',   p.attributes ->> 'brand',
    'price',   (p.attributes ->> 'price')::numeric,
    'in_stock', (p.attributes -> 'in_stock')::boolean
) AS product_json
FROM products p;
```

Example 20: JSONB in CTEs for complex analytics

```
-- Purchase funnel analysis
WITH purchase_stats AS (
    SELECT
        user_id,
        COUNT(*) FILTER (WHERE event_type = 'search') AS searches,
        COUNT(*) FILTER (WHERE event_type = 'pageview') AS pageviews,
        COUNT(*) FILTER (WHERE event_type = 'purchase') AS purchases,
        SUM((payload ->> 'amount')::numeric) FILTER (WHERE event_type = 'purchase')
AS total_spent
    FROM user_events
    GROUP BY user_id
)
SELECT
    user_id,
```

```

    searches,
    pageviews,
    purchases,
    total_spent,
    jsonb_build_object(
        'conversion_rate', ROUND(purchases::numeric / NULLIF(searches + pageviews,
0), 3),
        'avg_order_value', ROUND(total_spent / NULLIF(purchases, 0), 2)
    ) AS metrics
FROM purchase_stats;

```

Example 21: JSONB path operators on arrays

```

-- Find products where ANY color is 'black'
SELECT name FROM products
WHERE attributes @> '{"colors": ["black"]}';

-- Find events with specific nested filter
SELECT user_id
FROM user_events
WHERE event_type = 'search'
    AND payload @> '{"filters": {"price_max": 400}}';

```

Example 22: Indexing-friendly containment patterns

```

-- This USES a GIN index (efficient)
SELECT * FROM products WHERE attributes @> '{"brand": "Apple"}';

-- This does NOT use a GIN index (cast breaks it)
SELECT * FROM products WHERE (attributes ->> 'price')::numeric > 500;

-- For numeric range queries on JSONB, create a functional index:
CREATE INDEX idx_products_price ON products
    ((( attributes ->> 'price')::numeric));

-- Now this uses the functional index:
SELECT * FROM products
WHERE (attributes ->> 'price')::numeric BETWEEN 300 AND 1000;

```

JSONB Functions

jsonb_build_object — Construct JSONB from key-value pairs

```

SELECT jsonb_build_object(
    'user_id', 101,
    'action', 'login',
    'metadata', jsonb_build_object('ip', '192.168.1.1', 'ua', 'Mozilla/5.0')
);

```

```
-- {"user_id": 101, "action": "login", "metadata": {"ip": "192.168.1.1", "ua": "Mozilla/5.0"}}
```

jsonb_agg — Aggregate rows into a JSONB array

```
-- Get all products per category as a JSONB array
SELECT
  attributes ->> 'category' AS category,
  jsonb_agg(jsonb_build_object(
    'id',    id,
    'name',  name,
    'price', (attributes ->> 'price')::numeric
  ) ORDER BY (attributes ->> 'price')::numeric) AS products
FROM products
GROUP BY attributes ->> 'category';
```

jsonb_each / jsonb_each_text — Expand JSONB object to rows

```
-- See all top-level keys and values for iPhone
SELECT key, value
FROM products,
  jsonb_each(attributes)
WHERE name = 'iPhone 15 Pro';

-- Get specs as key-value pairs
SELECT key, value
FROM products,
  jsonb_each_text(attributes -> 'specs')
WHERE name = 'Dell XPS 15';
-- Returns: ram-16GB, storage-512GB, cpu-Intel i7-13700H, gpu-RTX 4060
```

jsonb_path_query — JSONPath evaluation

```
-- Find all products where price > 500 using JSONPath
SELECT name, jsonb_path_query(attributes, '$.price') AS price
FROM products
WHERE jsonb_path_exists(attributes, '$.price ? (@ > 500)');

-- Extract all tag values
SELECT name, jsonb_path_query_array(attributes, '$.tags[*]') AS tags
FROM products;
```

jsonb_set — Update nested value

```
-- Syntax: jsonb_set(target, path, new_value, create_missing=true)
UPDATE products
SET attributes = jsonb_set(
```

```

    attributes,
    '{ratings,avg}',
    '4.9'::jsonb,
    false -- don't create if missing
)
WHERE name = 'iPhone 15 Pro';

```

jsonb_insert — Insert into array

```

-- Insert a new color at position 0
UPDATE products
SET attributes = jsonb_insert(
    attributes,
    '{colors, 0}',
    '"gold"'::jsonb
)
WHERE name = 'iPhone 15 Pro';

```

jsonb_strip_nulls — Remove null values

```

SELECT jsonb_strip_nulls('{ "a": 1, "b": null, "c": { "d": null, "e": 2 } }'::jsonb);
-- Returns: { "a": 1, "c": { "e": 2 } }

```

jsonb_typeof — Get type of JSONB value

```

SELECT
    name,
    jsonb_typeof(attributes -> 'price') AS price_type,    -- number
    jsonb_typeof(attributes -> 'tags') AS tags_type,     -- array
    jsonb_typeof(attributes -> 'specs') AS specs_type,   -- object
    jsonb_typeof(attributes -> 'in_stock') AS stock_type -- boolean
FROM products;

```

GIN Indexing for JSONB

GIN Index Structure for JSONB column:

GIN Index on products.attributes	
Entry tree (inverted index):	
"brand"	→ {row 1, row 2, row 3}
"Apple"	→ {row 1}
"Samsung"	→ {row 2}
"category"	→ {row 1, row 2, row 3}

"smartphone"	→ {row 1, row 2}	
"laptop"	→ {row 4}	
"in_stock"	→ {row 1, row 2, row 4}	
true	→ {row 1, row 2, row 4}	

Two GIN operator classes

```
-- 1. jsonb_ops (default) – indexes every key and value at all levels
CREATE INDEX idx_products_attrs_gin ON products USING GIN (attributes);
-- Supports: @>, <@, ?, ?|, ?&

-- 2. jsonb_path_ops – indexes only values, not keys (smaller, faster for @>)
CREATE INDEX idx_products_attrs_path ON products USING GIN (attributes
jsonb_path_ops);
-- Supports ONLY: @> (not ?, ?|, ?&)

-- Check index usage
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM products WHERE attributes @> '{"brand": "Apple"}';
```

Partial GIN index (advanced)

```
-- Index only in-stock products (much smaller index)
CREATE INDEX idx_products_instock ON products USING GIN (attributes)
WHERE (attributes -> 'in_stock')::boolean = true;
```

Composite indexing strategy

```
-- For queries filtering on category + JSONB
CREATE INDEX idx_products_cat ON products ((attributes ->> 'category'));
CREATE INDEX idx_products_price ON products (((attributes ->> 'price')::numeric));
CREATE INDEX idx_products_gin ON products USING GIN (attributes jsonb_path_ops);
```

JSONPath Queries

JSONPath is a standard language for navigating JSON structures, supported in PostgreSQL 12+.

```
-- Basic path: $.key
SELECT jsonb_path_query('{ "a": 1, "b": 2 }', '$.a'); -- 1

-- Array access: $.array[n]
SELECT jsonb_path_query('{ "items": [10,20,30] }', '$.items[1]'); -- 20

-- Wildcard: $.array[*]
SELECT jsonb_path_query_array('{ "tags": ["a","b","c"] }', '$.tags[*]');
```

```

-- Filter expression: $.key?(condition)
SELECT jsonb_path_exists(attributes, '$.price ? (@ > 1000)')
FROM products;

-- Recursive descent: $.key
SELECT jsonb_path_query_array(
    '{"a": {"b": 1}, "c": {"b": 2}}'::jsonb,
    '$..b'
); -- [1, 2]

-- String methods in JSONPath
SELECT jsonb_path_query_array(
    attributes,
    '$.tags[*] ? (@ starts with "p")'
)
FROM products;
-- Returns tags starting with 'p': ["premium","productivity"]

-- Arithmetic in JSONPath
SELECT jsonb_path_query(attributes, '$.price * 0.9') -- 10% discount
FROM products WHERE name = 'iPhone 15 Pro';

-- Date/time in JSONPath (PostgreSQL 14+)
SELECT jsonb_path_query(
    '{"ts": "2024-01-15T10:30:00Z"}'::jsonb,
    '$.ts.datetime()'
);

```

jsonb_path_match — Boolean path result

```

-- All products where rating > 4.6
SELECT name FROM products
WHERE jsonb_path_match(attributes, '$.ratings.avg > 4.6');

```

Modifying JSONB Data

```

-- 1. Merge/overwrite keys using || operator
UPDATE products
SET attributes = attributes || '{"in_stock": true, "last_updated": "2024-01-15"}'
WHERE name = 'Sony WH-1000XM5';

-- 2. Delete a key using - operator
UPDATE products
SET attributes = attributes - 'last_updated';

-- 3. Delete multiple keys
UPDATE products
SET attributes = attributes - ARRAY['last_updated', 'warranty_years'];

```

```

-- 4. Delete by path using #- operator
UPDATE products
SET attributes = attributes #- '{specs,gpu}'
WHERE name = 'Dell XPS 15';

-- 5. Append to array
UPDATE products
SET attributes = jsonb_set(
    attributes,
    '{colors}',
    (attributes -> 'colors') || '"gold"'
)
WHERE name = 'iPhone 15 Pro';

```

Real-World Use Cases

1. Feature Flags / Configuration Store

```

CREATE TABLE feature_flags (
    id          SERIAL PRIMARY KEY,
    name        TEXT UNIQUE NOT NULL,
    config      JSONB NOT NULL DEFAULT '{}'
);

INSERT INTO feature_flags VALUES
(1, 'new_checkout', '{"enabled": true, "rollout_pct": 50, "allowed_countries": ["US", "CA"]}'),
(2, 'dark_mode',    '{"enabled": true, "rollout_pct": 100}'),
(3, 'beta_features', '{"enabled": false, "rollout_pct": 0, "whitelist_users": [101,102,103]}');

-- Check if user 101 is in beta features whitelist
SELECT name, config ->> 'rollout_pct' AS pct
FROM feature_flags
WHERE config @> '{"enabled": true}'
      AND (config ->> 'rollout_pct')::int > 0;

```

2. Audit Log

```

CREATE TABLE audit_log (
    id          BIGSERIAL PRIMARY KEY,
    table_name  TEXT,
    operation   TEXT,
    row_id      BIGINT,
    old_data    JSONB,
    new_data    JSONB,
    changed_by  TEXT,
    changed_at  TIMESTAMPTZ DEFAULT now()
);

```

```

-- Generic audit trigger
CREATE OR REPLACE FUNCTION audit_trigger_fn()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO audit_log (table_name, operation, row_id, old_data, new_data,
changed_by)
    VALUES (
        TG_TABLE_NAME,
        TG_OP,
        COALESCE(NEW.id, OLD.id),
        CASE WHEN TG_OP != 'INSERT' THEN row_to_json(OLD)::jsonb END,
        CASE WHEN TG_OP != 'DELETE' THEN row_to_json(NEW)::jsonb END,
        current_user
    );
    RETURN COALESCE(NEW, OLD);
END;
$$;

```

3. Multi-tenant Schema Customization

```

CREATE TABLE tenant_records (
    id          BIGSERIAL PRIMARY KEY,
    tenant_id   INT NOT NULL,
    record_type TEXT NOT NULL,
    standard_fields JSONB NOT NULL, -- shared across tenants
    custom_fields  JSONB DEFAULT '{}' -- per-tenant customizations
);

-- Query combining standard and custom fields
SELECT
    id,
    standard_fields ->> 'name'      AS name,
    standard_fields ->> 'email'    AS email,
    custom_fields   ->> 'crm_ref'   AS crm_reference,
    custom_fields   ->> 'account_mgr' AS account_manager
FROM tenant_records
WHERE tenant_id = 42
    AND standard_fields @> '{"status": "active"}';

```

Common Mistakes

- 1. Casting for comparisons** — `attributes ->> 'price' > '500'` compares as text, not number. Always cast: `(attributes ->> 'price')::numeric > 500`.
- 2. **Using `->` when you need `->>` **** — `attributes -> 'brand' = 'Apple'` fails because `->` returns jsonb, not text. Use `attributes ->> 'brand' = 'Apple'`.
- 3. Forgetting GIN index doesn't help range queries** — `(attributes ->> 'price')::numeric > 500` won't use GIN. Create a functional B-tree index for range queries.

4. **Overly fat JSONB columns** — Storing frequently-queried, always-present fields as JSONB adds overhead. Put them in proper columns.
5. **Not quoting strings in jsonb literals** — `'{"key": value}'` is invalid JSON; must be `'{"key": "value"}'` .
6. **Using json instead of jsonb** — `json` type is almost never what you want; it doesn't support indexing or most operators.
7. **Mutating JSONB in loops** — Repeatedly calling `jsonb_set()` in application code for many updates is slow; batch updates instead.

Best Practices

1. **Use jsonb_path_ops GIN class** when you only need `@>` containment — it's smaller and faster than default `jsonb_ops` .
2. **Normalize hot fields** — Extract high-cardinality, frequently-filtered fields (like `user_id` , `created_at` , `status`) into proper columns. Keep rare/variable attributes in JSONB.
3. **Partial indexes** — For large tables where you only query a subset (e.g., only active records), use partial GIN indexes.
4. **Validate schema with CHECK constraints** — Add constraints to ensure required keys exist:

```
ALTER TABLE products ADD CONSTRAINT chk_attrs
CHECK (attributes ? 'brand' AND attributes ? 'price');
```

5. **Use jsonb_strip_nulls** before storing to keep documents lean.
6. **Consider generated columns** for frequently-accessed JSONB subfields:

```
ALTER TABLE products
ADD COLUMN price_col NUMERIC
GENERATED ALWAYS AS ((attributes ->> 'price')::numeric) STORED;
```

Performance Considerations

```
-- EXPLAIN to check GIN index usage
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT * FROM products WHERE attributes @> '{"brand": "Apple"}';

-- Index size comparison
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE relname = 'products';
```

```
-- Monitor JSONB query performance
SELECT query, mean_exec_time, calls
FROM pg_stat_statements
WHERE query LIKE '%attributes%'
ORDER BY mean_exec_time DESC;
```

Key insight: GIN indexes have higher maintenance cost than B-tree. For write-heavy tables, consider `gin_pending_list_limit` tuning and `fastupdate = off` if you notice index bloat.

Interview Questions & Answers

Q1: What is the difference between `json` and `jsonb` in PostgreSQL?

A: `json` stores the raw text verbatim — duplicate keys are preserved, key order is maintained, and whitespace is kept. It re-parses on every read. `jsonb` stores a decomposed binary representation — it deduplicates keys (keeping last value), sorts keys alphabetically, strips whitespace, but reads much faster and supports GIN indexing. Use `jsonb` in virtually all cases; `json` only when you need exact text round-tripping.

Q2: Explain the `@>` operator and why it's important for JSONB.

A: `@>` is the containment operator: `a @> b` returns true if `a` contains all the key-value pairs (or array elements) in `b`. For example, `'{"a":1,"b":2}'::jsonb @> '{"a":1}'::jsonb` is true. It's critical because it's the primary operator supported by GIN indexes, enabling fast existence/equality checks on nested JSON values without full table scans.

Q3: When should you use `jsonb_ops` vs `jsonb_path_ops` GIN index?

A: `jsonb_ops` indexes every key and value, supporting `@>`, `<@`, `?`, `?|`, `?&`. `jsonb_path_ops` indexes only values (not keys), so it's smaller and faster for `@>` queries. Choose `jsonb_path_ops` when you only need containment queries; use `jsonb_ops` when you also need key-existence queries (`?`, `?|`, `?&`).

Q4: How would you efficiently query products where the price (stored in JSONB) is between 300 and 800?

A: GIN indexes don't support range queries on JSONB values. Create a functional B-tree index: `CREATE INDEX ON products (((attributes ->> 'price')::numeric));`. Then query `WHERE (attributes ->> 'price')::numeric BETWEEN 300 AND 800` to use it. Alternatively, consider a generated column for frequently-queried numeric fields.

Q5: Explain JSONPath and how it differs from the `->` / `#>` operators.

A: JSONPath is a full query language (SQL/JSON standard) supporting filters, arithmetic, recursive descent, and string methods. `->` and `#>` are simple navigation operators. `JSONPath ( jsonb_path_query , jsonb_path_exists , jsonb_path_match )` enables expressions like `$.items[*] ? (@.price > 100)` to filter array elements, which is impossible with simple navigation operators.

Q6: How does JSONB handle arrays in containment checks?

A: For arrays, `@>` checks if all elements in the right-side array exist in the left-side array (not necessarily in order). `'[1,2,3]'::jsonb @> '[3,1]'::jsonb` returns true. This is set-based containment, not positional. This makes it useful for "has all these tags" queries.

Q7: What are the trade-offs of storing too much in JSONB vs using relational columns?

A: JSONB advantages: flexibility for variable schemas, no schema migrations for new fields, easier to store tree structures. Disadvantages: no type enforcement, range queries need functional indexes, aggregations require casts, JOIN performance on JSONB fields is worse than on native columns, foreign key constraints aren't possible on JSONB fields, and table bloat from TOAST storage.

Q8: How would you implement a schema migration that adds a new required field to existing JSONB documents?

A: Use a backfill UPDATE: `UPDATE table SET data = data || '{"new_field": "default_value"}' WHERE NOT data ? 'new_field';` . For large tables, do this in batches with a CTE or procedural loop to avoid long-running transactions. Add a CHECK constraint afterward to enforce the field's presence.

Q9: Explain the performance implications of JSONB's TOAST storage.

A: JSONB documents larger than ~2KB are compressed and moved to a TOAST table. Reads require a separate heap access. Writes that modify even a small part of a large JSONB document must rewrite the entire value. For documents approaching TOAST threshold, consider splitting into multiple columns or separate tables.

Q10: How do you safely update a single nested key in a JSONB column?

A: Use `jsonb_set(target, path_array, new_value)` : `UPDATE t SET data = jsonb_set(data, '{nested,key}', 'new_val')` WHERE id = 1; . Never do string concatenation or full-document replacement. The `||` merge operator replaces the entire top-level key, so for deep updates, `jsonb_set` is required.

Q11: What monitoring queries do you use to find slow JSONB queries in production?

A: Query `pg_stat_statements` : `SELECT query, mean_exec_time, calls FROM pg_stat_statements WHERE query LIKE '%->%' OR query LIKE '%@>%' ORDER BY mean_exec_time DESC;` . Also check `EXPLAIN (ANALYZE, BUFFERS)` to confirm GIN index usage. Look for `Seq Scan` on large JSONB tables as a red flag.

Q12: How does PostgreSQL's `jsonb_path_ops` GIN index achieve its smaller size?

A: `jsonb_path_ops` stores hashes of the complete path from root to each leaf value (e.g., a hash of `{brand: "Apple"}`), rather than indexing each key and value independently. This means fewer, smaller index entries. The trade-off is it can only be used for `@>` (path-based containment) and cannot answer "does key X exist?" queries.

Exercises and Solutions

Exercise 1

Create a `users` table with a JSONB `preferences` column. Insert 5 users. Write a query to find all users who have `notifications.email` set to `true` and have the `theme` key set to `"dark"` .

Solution:

```
CREATE TABLE users (  
  id          SERIAL PRIMARY KEY,  
  username    TEXT NOT NULL,  
  preferences JSONB DEFAULT '{}'  
);
```

```

INSERT INTO users (username, preferences) VALUES
('alice', '{"theme": "dark", "notifications": {"email": true, "sms": false}}'),
('bob',   '{"theme": "light", "notifications": {"email": true, "sms": true}}'),
('carol', '{"theme": "dark", "notifications": {"email": false, "sms": false}}'),
('dave',  '{"theme": "dark", "notifications": {"email": true, "sms": false}}'),
('eve',   '{"theme": "system", "notifications": {"email": true, "sms": true}}');

-- Solution using @> containment
SELECT username FROM users
WHERE preferences @> '{"theme": "dark", "notifications": {"email": true}}';
-- Returns: alice, dave

```

Exercise 2

Given the `products` table, write a query that returns a JSON object per category with the min price, max price, and array of product names, all computed dynamically.

Solution:

```

SELECT
  attributes ->> 'category' AS category,
  jsonb_build_object(
    'min_price', MIN((attributes ->> 'price')::numeric),
    'max_price', MAX((attributes ->> 'price')::numeric),
    'products', jsonb_agg(name ORDER BY (attributes ->> 'price')::numeric)
  ) AS summary
FROM products
GROUP BY attributes ->> 'category';

```

Exercise 3

Add a GIN index to the `products.attributes` column using `jsonb_path_ops`, then verify with `EXPLAIN` that a containment query uses it.

Solution:

```

CREATE INDEX idx_products_path ON products
USING GIN (attributes jsonb_path_ops);

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM products WHERE attributes @> '{"category": "smartphone"}';
-- Should show: Index Scan using idx_products_path on products

```

Cross-References

- [07_indexes](#) — GIN index internals and operator classes
- [02_full_text_search.md](#) — GIN indexes for tsvector (same index structure, different use)
- [04_query_optimization](#) — EXPLAIN analysis for JSONB queries
- [05_extensions_overview.md](#) — `pg_trgm` extension (also uses GIN)
- [09_stored_procedures_functions.md](#) — Trigger-based JSONB audit patterns

- **PostgreSQL Docs** — <https://www.postgresql.org/docs/current/functions-json.html>